

Nombre:	Documento:
---------	------------

- Durante el examen no se responden preguntas y tiene 1 hora y 45 minutos para resolverlo.
- No puede usar aparatos electrónicos y el examen es individual.
- **Por favor disfrutar el examen y tener confianza en sus conocimientos**
- Tenga en cuenta: “*Qué hubiera sido, si antes ...*” Dario Gómez

Ejercicio 1. (2 puntos) Resolver los siguientes ejercicios:

- ✓ Verdadero o falso: $n^{5/2}$ es $O(n^2 \ln n)$.
- ✓ Hallar el valor de $Nidiea([1, 2, 3, 4])$. Hallar la complejidad computacional de dicho algoritmo. Para esto tener en cuenta únicamente las asignaciones y operaciones aritméticas

```
def Niidea(S):
    n = len(S)
    total = 0

    for j in range(0, n, 2):
        for i in range(0, 2):
            total = total + 2*S[j]*S[i]
    return total
```

Ejercicio 2. (2 puntos) Resolver los siguientes ejercicios:

- ✓ Escriba una función recursiva en python de nombre $MinMax(L)$ que recibe una lista de números L no vacía y retorna el máximo y el mínimo de una lista.
- ✓ Escribir una función recursiva $Picos(L)$ que cuente el número de picos en la lista L . Un pico se encuentra si el número a la izquierda y derecha son estrictamente menores.

Ejercicio 3. (1 puntos) Considerar la documentación de $ArrayStack$ para el siguiente ejercicio:

- ✓ Escribir una función $Contar(S, S1, a)$ que recibe dos Stacks S , $S1$ y un elemento a . Dicha función debe retornar el número de veces que aparece a en S y $S1$.
Nota: No puede crear listas ni stacks adicionales. Puede crear variables simples y solo puede usar los métodos públicos del stack. Además los stack deben quedar en la configuración original.

```

class Empty(Exception):
    pass

class ArrayStack:

    def __init__(self):
        """Create an empty stack."""
        self._data = []                # nonpublic list instance

    def __len__(self):
        """Return the number of elements in the stack."""
        return len(self._data)

    def is_empty(self):
        """Return True if the stack is empty."""
        return len(self._data) == 0

    def push(self, e):
        """Add element e to the top of the stack."""
        self._data.append(e)          # new item stored at end of list

    def top(self):
        """Return (but do not remove) the element at the top of the stack.

        Raise Empty exception if the stack is empty.
        """
        if self.is_empty():
            raise Empty('Stack is empty')
        return self._data[-1]         # the last item in the list

    def pop(self):
        """Remove and return the element from the top of the stack (i.e., LIFO).

        Raise Empty exception if the stack is empty.
        """
        if self.is_empty():
            raise Empty('Stack is empty')
        return self._data.pop()       # remove last item from list

```